Build your own AI news agent

with Redis + LangGraph.js

- Guy Royse

SENIOR DEVELOPER
ADVOCATE, REDIS

Socials

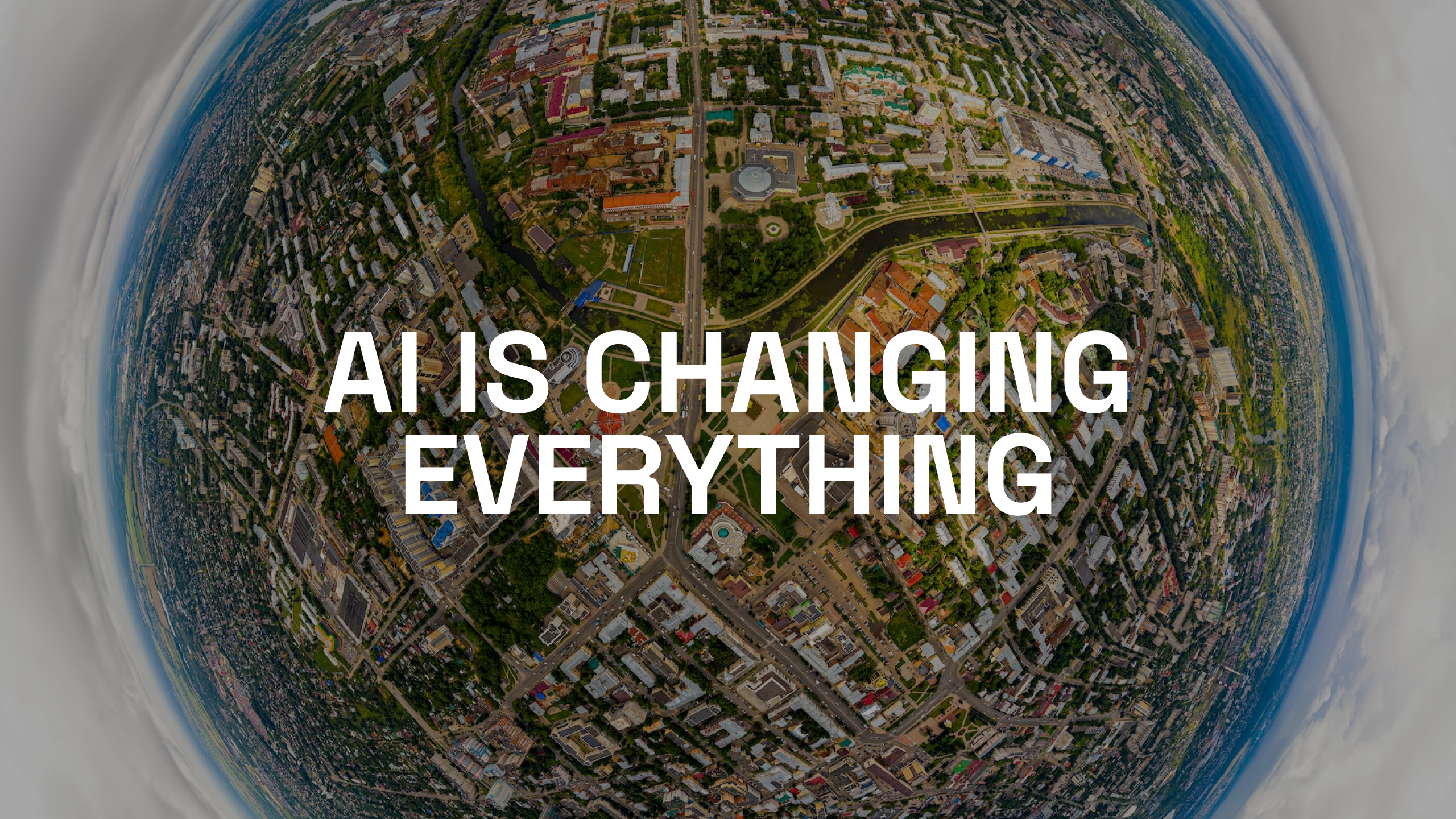
@guyroyse

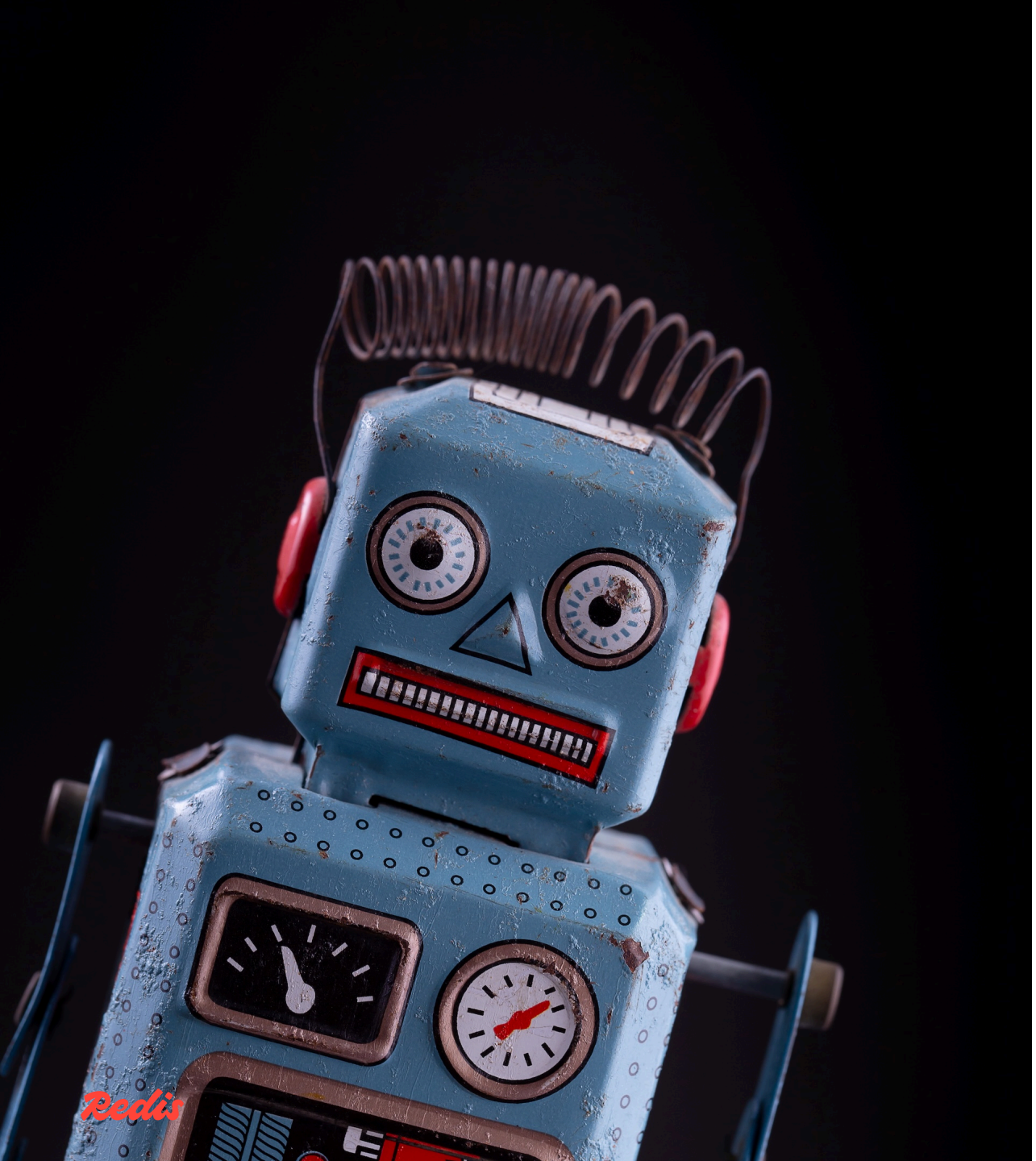
github.com/guyroyse

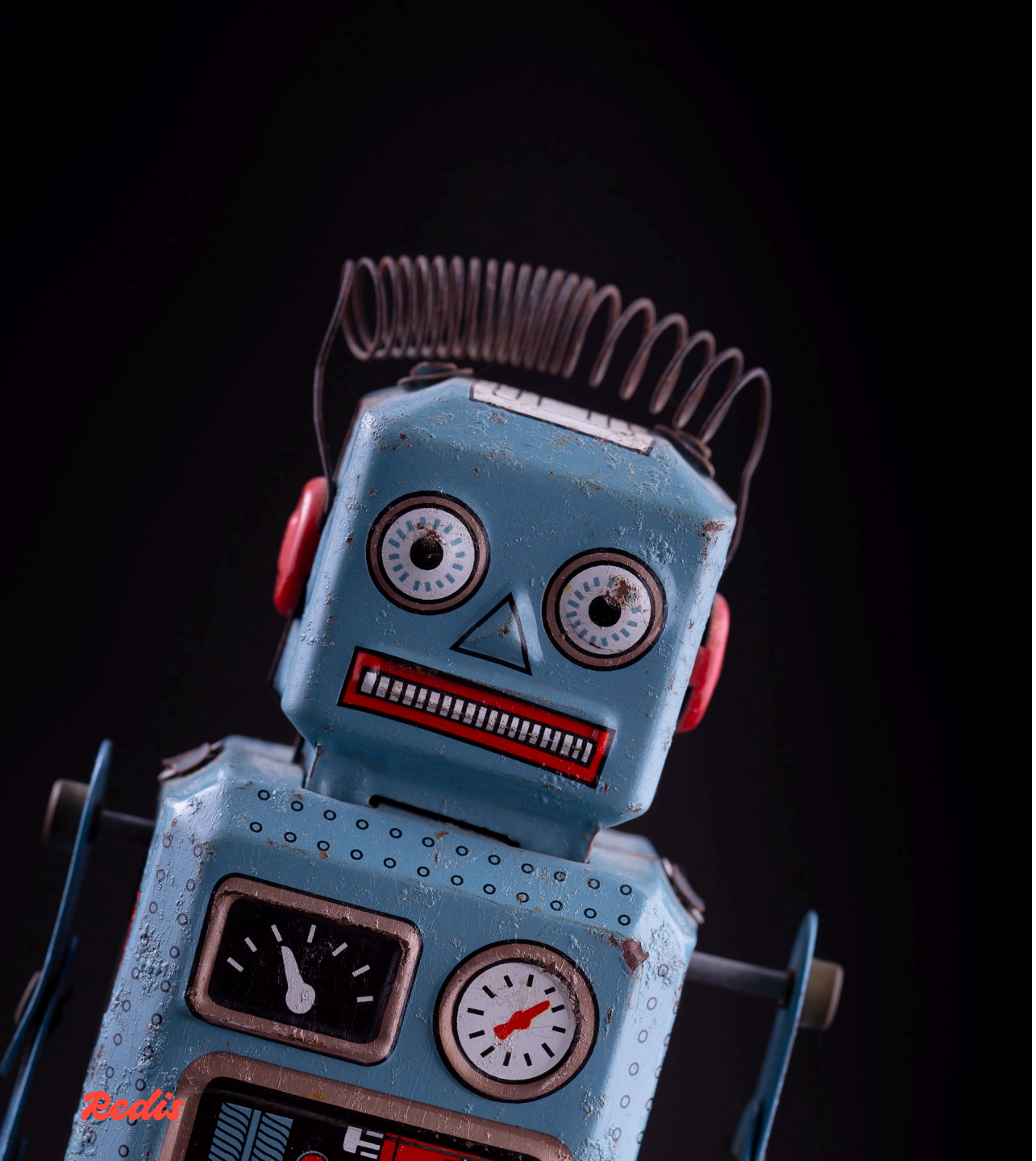
@guy.dev

guy.dev

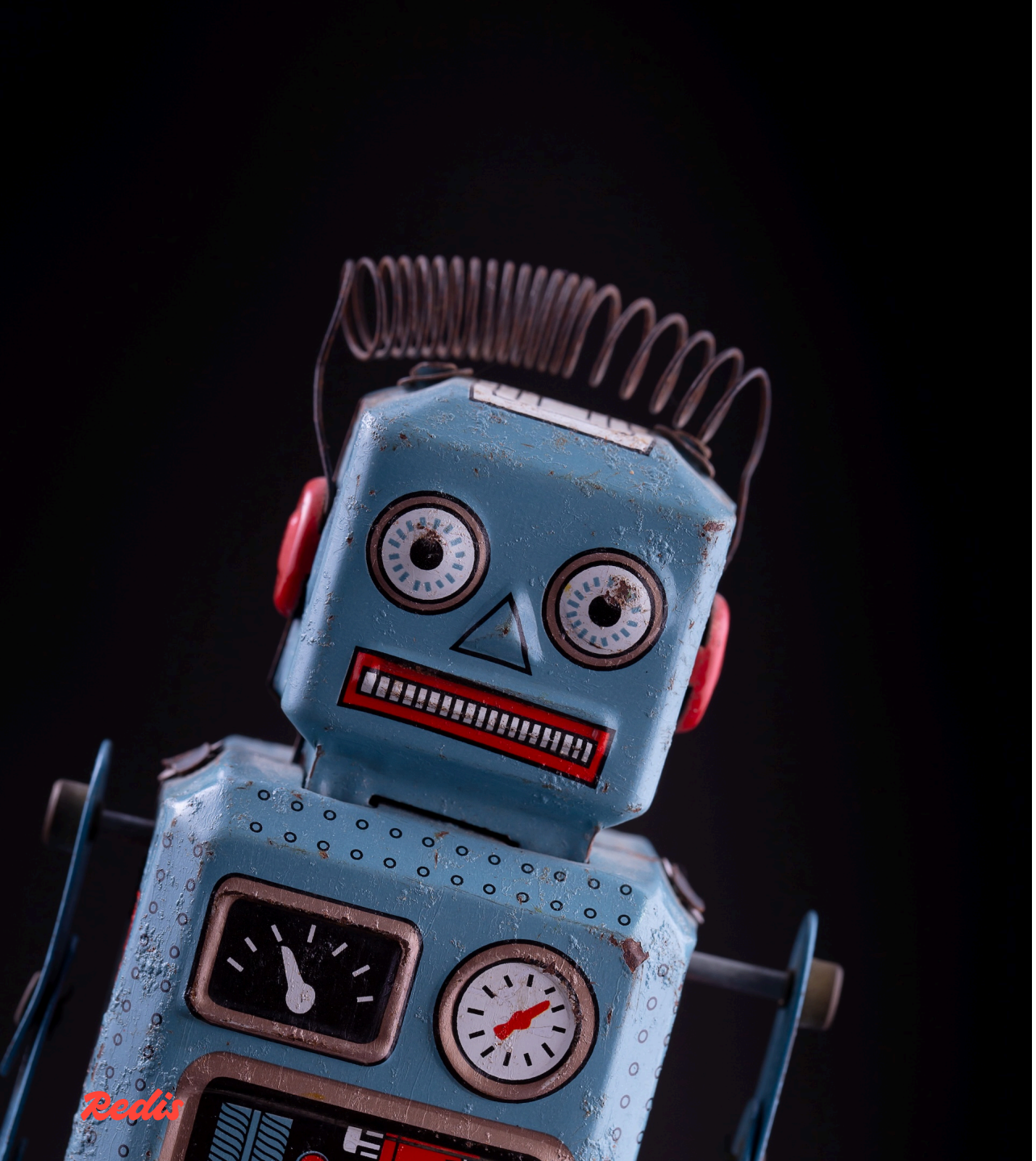
Redis


AI IS CHANGING
EVERYTHING


Redis



Building apps with AI.



Building apps with AI.

Build apps that ***use*** AI.

Tools we're using today

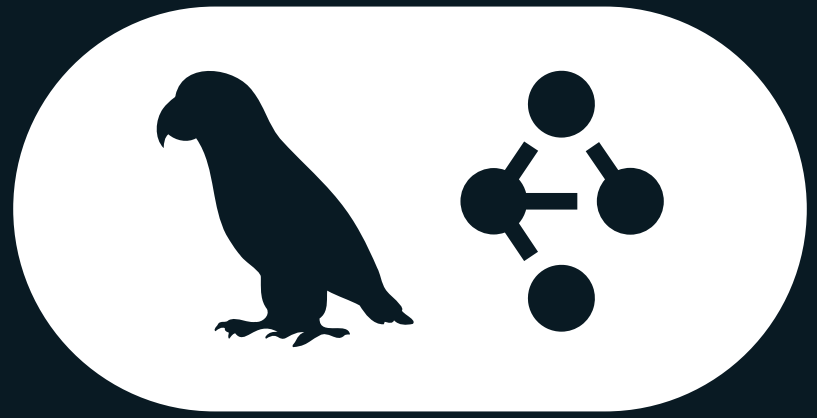
LangGraph.js

Workflow Orchestration

- Graph-based
- Nodes define the work
- Edges define the flow

Custom Annotations

- Define the state the workflow updates



Redis

Redis

Redis Search

Semantic Search

Agent Memory Server

WHAT ARE WE BUILDING TODAY?

Redis

Search

Sources

- [ ] bbc news

Date Range

From

mm/dd/yyyy

To

mm/dd/yyyy

Topics

Type to search topics...

People

Type to search people...

Organizations

Type to search organizations...

Locations

Type to search locations...

Semantic Search

Search by meaning...

Search

Activity

Vasilis Palmas Winston Churchill

PM Stands by Decision Not to Join Strikes on Iran and Sends More Jets to Qatar Sir Keir Starmer has said he stands by his decision not to join the initial US-Israel strikes on Iran on Saturday, and said talks would be the best way forward. At a Downing Street news conference, the PM said the UK had "the strength to stand by our values and our principles no matter the pressure to do otherwise"....

BBC News Apr 29, 2:21 PM

Iran school and nearby military base struck multiple times, satellite image reveals

Mar 5, 2026

BBC Verify Geopolitics Human Rights Human Rights News Agency Iran Iranian Revolutionary Guard Corps Jamon Van den Hoek McKenzie Intelligence Services Military Conflict Minab N R Jenzen Jones Oregon Oz Smith Pete Hegseth Tasnim Tehran Zanjan

IRAN SCHOOL AND NEARBY MILITARY BASE STRUCK MULTIPLE TIMES, SATELLITE IMAGE REVEALS Satellite imagery analysis reveals multiple strikes and burn marks around a school in southern Tehran, suggesting it was hit more than once, in an attack Iranian authorities say killed 168 people on Saturday. Verified videos and satellite imagery show extensive damage around the Shajareh...

BBC News Apr 29, 2:21 PM

Starmer tries to reassure as Iran crisis deepens

Mar 5, 2026

Afghanistan Cyprus Economy International Relations Middle East Oman Politics Rachel Reeves Sir Keir Starmer UK UK government

STARMER TRIES TO REASSURE AS IRAN CRISIS DEEPENS "Worried sick". They are not the words a prime minister wants to be using from the Downing Street podium to describe the emotional state of their country. Sir Keir Starmer identified three ways in which the public are worrying: about their family and friends, about the impact of this war on the British economy, and about the potential for greater...

Ingested 5 of 74 articles Apr 29, 2:28 PM

> BBC News (5)

Ingested 1 of 75 articles Apr 30, 5:05 PM

> BBC News (1)

Chat

So what's up with the news today?

Here's a look at some of the latest news stories:

1. **Meningitis Outbreak in Kent**: A meningitis outbreak in Kent has led to 20 reported cases, raising concerns about the rapid spread and potential increased invasiveness of the bacteria. Experts are investigating the outbreak, which may be linked to a nightclub, and are considering factors like vaping and environmental conditions as possible contributors. Read more
2. **Modern Slavery and Forced Marriage Allegations**: Nine people have been arrested in Cheshire following allegations of serious sexual offenses, forced marriage, and modern slavery within a religious group known as the Ahmadi Religion of Peace and Light. The investigation is focused on the alleged crimes rather than the group's religious beliefs. Read more
3. **Stabbing in Golders Green**: Shloime Rand, a victim of a stabbing attack in Golders Green, has spoken out about his experience and the need for action against antisemitism. In response, the UK government has announced additional funding for security measures in Jewish communities. Read more
4. **Iran Crisis and UK Response**: Sir Keir Starmer has addressed public concerns over the Iran crisis, highlighting the challenges of evacuating British nationals and the potential economic impacts. He reassured the public of ongoing efforts to manage the situation. Read more
5. **Golders Green Attack Details**: A video has surfaced detailing the attack on two Jewish men in Golders Green, which has been classified as a terrorist incident. Both victims are currently in stable condition. Read more

Let me know if you'd like more details on any of these stories!

Type a message and press Enter...

Four components

From raw feeds to personalized briefs

	NAME	WHAT WE'RE BUILDING
1	Ingestion	Process RSS feeds into enriched articles and store them in Redis
2	Search	Search articles by topic, entity, and meaning with Redis Search
3	Chat	Converse about the news and extract memories with Agent Memory Server
4	Brief	Generate personalized news briefs

Redis

LET’S GET SET UP

Two options

Option A: Remote virtual environment

- No install needed
- Scan the QR code and follow the instructions

Option B: Local Docker environment

- Runs locally in Docker
- Clone the repo and run `docker compose up`

Redis

Remote virtual environment

Scan the QR code

Follow the instructions to get your environment set up.

You will be asked for you email address.
You'll be emailed a link to your environment so don't lie.

Redis

Local Docker environment

Clone this if you're using Docker

```
$ git clone git@github.com:redis-developer/
            ai-news-agent-lang-graph-js-redis-workshop.git
$ docker compose up -d
```

Open `localhost` in your browser.

Redis

The cheat code

If you get stuck...

github.com/redis-developer/ai-news-agent-lang-graph-js-redis-demo

Redis

INSTRUCTIONS

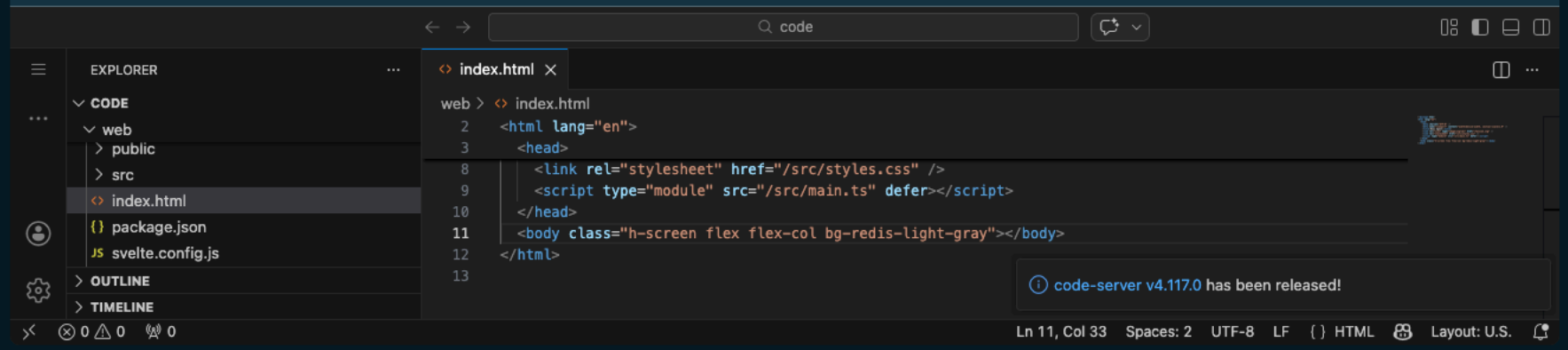
Redis + LangGraph.js Workshop

Build an AI-powered news agent with Redis and LangGraph.js

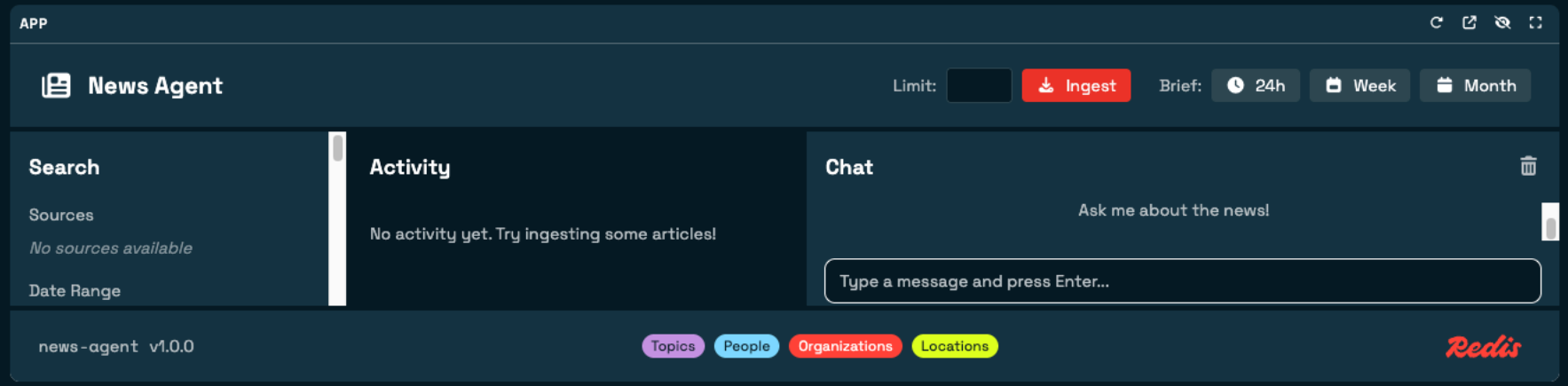
LangGraph.js workflows for ingestion, chat, and briefs
Redis Search for hybrid vector + structured queries
Agent Memory Server for short and long-term memory

Get Started

CODE



APP



TERMINAL

```
[web] > @news-agent/web@1.0.0 dev
[web] > vite --port 5173
[web]
[server] [nodemon] 3.1.14
[server] [nodemon] to restart at any time, enter `rs`
[server] [nodemon] watching path(s): *.*
[server] [nodemon] watching extensions: ts,json
[server] [nodemon] starting `tsx src/server.ts`
[web]
[web]   VITE v7.3.1  ready in 1076 ms
[web]
[web]   ➜  Local:   http://localhost:5173/app/
[web]   ➜  Network: http://172.25.0.7:5173/app/
[server] Creating index news:aggregator:article:index
[server] Index news:aggregator:article:index created
[server] NOTE: If you change the index in code, you must delete the index in Redis CLI with: FT.DROPINDEX news:aggregator:article:index
[server] Server running at http://localhost:3000
```

STAGE 1: FEED INGESTION

What we’re building

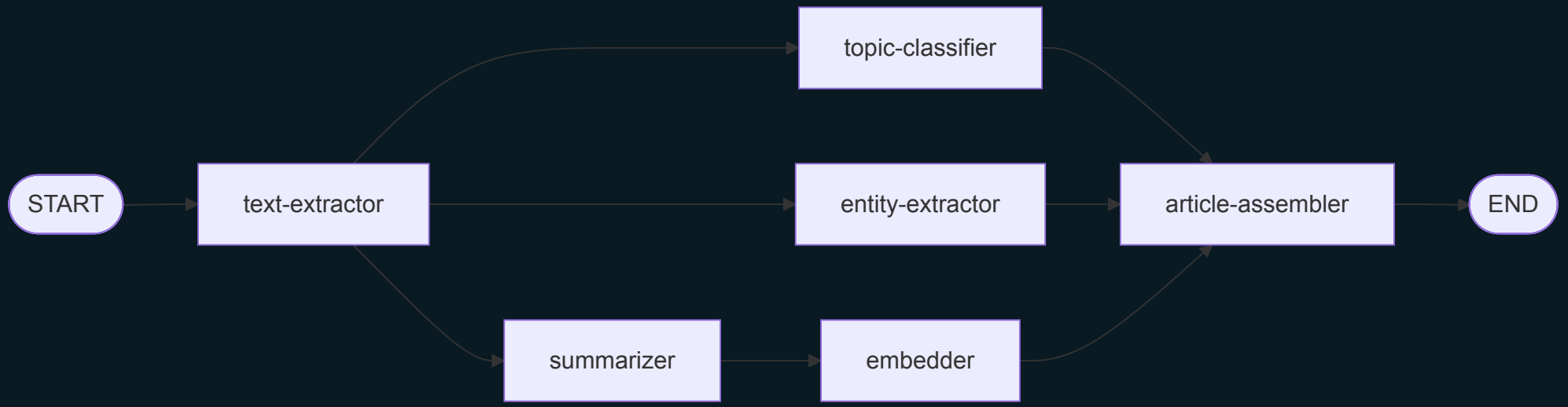
An ingestion pipeline

- Fetch RSS feeds
- Extract clean text from HTML
- Summarize, classify topics, extract entities
- Generate vector embeddings
- Assemble and store in Redis

Redis

The final graph

What you'll build by the end of Stage 1



Redis

LANGGRAPH.JS BASICS

Redis

How LangGraph.js works

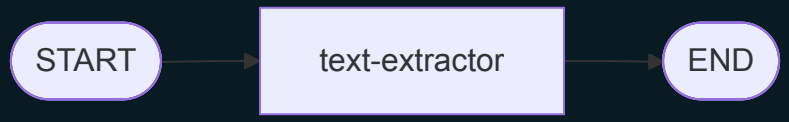
Four steps to every workflow

1. Define a **state** — the data flowing through the graph
2. Write **nodes** — functions that transform state
3. Connect with **edges** — define execution order
4. **Compile** and **invoke**

THE SIMPLEST GRAPH

A single node

START → text-extractor → END



Defining state

`Annotation.Root()` defines the shape

```
export const ArticleAnnotation = Annotation.Root({
  feedItem: Annotation<FeedItem>(),
  content: Annotation<string>(),
  article: Annotation<ArticleData>()
})
```

- Each field is a slot in the shared state
- Nodes read from and write to these slots

Redis

Writing a node

Functions that transform state

```
async function textExtractor(state: ArticleState): Promise<Partial<ArticleState>> {
  const { feedItem } = state

  const text = extractTextFromHtml(feedItem.html)
  const prompt = buildPrompt(text)
  const llm = fetchLLM()
  const response = await llm.invoke(prompt)

  return { content: response.content as string }
}
```

- Takes full state in, returns partial state out
- LangGraph.js merges the return into shared state

Redis

Fetching the LLM

Using LangChain.js

```
export function fetchLLM(): ChatOpenAI {
  return new ChatOpenAI({
    modelName: 'gpt-4o-mini',
    temperature: 0.3,
    apiKey: OPENAI_API_KEY
  })
}
```

Redis

Wiring the graph

Nodes + edges + compile

```
const graph = new StateGraph(ArticleAnnotation)

graph.addNode('text-extractor', textExtractor)

graph.addEdge(START, 'text-extractor')
graph.addEdge('text-extractor', END)

const workflow = graph.compile()
```

- Add nodes
- Connect with edges
- Compile

Redis

Invoking the workflow

Compile once, invoke many times

```
const initialState = { feedItem }
const finalState = await articleWorkflow.invoke(initialState)
console.log(finalState.article)
```

- Pass initial state to `invoke()`
- Get final state back when the graph completes

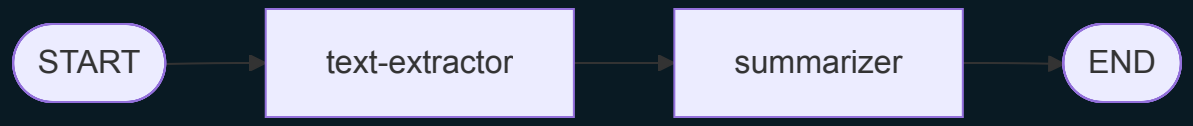
Redis

MULTI-NODE GRAPHS

Redis

Adding the summarizer

START → text-extractor → summarizer → END



Redis

The Summarizer Node

Same pattern as the text extractor

```
async function summarizer(state: ArticleState): Promise<Partial<ArticleState>> {
  const { content } = state

  const prompt = buildPrompt(content)
  const llm = fetchLLM()
  const response = await llm.invoke(prompt)

  return { summary: response.content as string }
}
```

- Reads `content` written by the text extractor
- Writes `summary` back to state

Redis

Sequential chaining

One node's output is the next node's input

```
graph.addNode('text-extractor', textExtractor)
graph.addNode('summarizer', summarizer)

graph.addEdge(START, 'text-extractor')
graph.addEdge('text-extractor', 'summarizer')
graph.addEdge('summarizer', END)
```

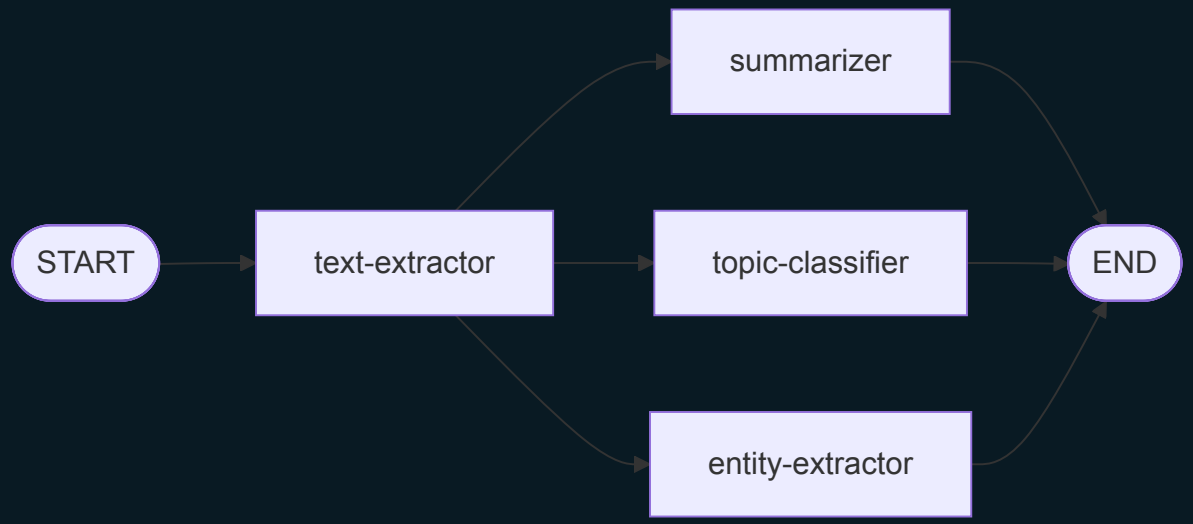
- Text extractor writes content to state
- Summarizer reads content from state

Redis

FAN-OUT: PARALLEL PROCESSING

Three branches from text-extractor

Summarizer, topic classifier, entity extractor — all at once



Redis

Fan-out is just multiple edges

Three edges from one node → runs all three **in parallel**

```
...
graph.addNode('text-extractor', textExtractor)
graph.addNode('summarizer', summarizer)
graph.addNode('topic-classifier', topicClassifier)
graph.addNode('entity-extractor', entityExtractor)

graph.addEdge(START, 'text-extractor')
graph.addEdge('text-extractor', 'summarizer')
graph.addEdge('text-extractor', 'topic-classifier')
graph.addEdge('text-extractor', 'entity-extractor')
...
```

Redis

Unstructured output

LLMs return strings — but you need data

”The topics are Technology, AI, and Business”

”The named entities are Mark Zuckerberg, Apple, and San Francisco”

What do you do?

- Give the LLM really explicit instructions?
- Parse it? Regex? Hope for the best?
- What if the LLM changes its formatting?

Redis

Structured output

Define a schema with Zod:

```
const topicsSchema = z.object({
  topics: z.array(z.string()).describe('Broad topics for the article')
})
```

Tell the LLM to use that schema:

```
const llm = fetchLLM().withStructuredOutput(topicsSchema)
const result = await llm.invoke(prompt)
```

Get validated JSON back:

```
{ "topics": ["Technology", "Artificial Intelligence"] }
```

No parsing, no regex, no guessing

Redis

The topic classifier node

Structured output in action

```
async function topicClassifier(state: ArticleState): Promise<Partial<ArticleState>> {
  const { feedItem, content } = state

  const prompt = buildPrompt(feedItem.title, content)
  const llm = fetchLLM().withStructuredOutput(topicsSchema)
  const result = await llm.invoke(prompt)

  return { topics: result.topics }
}
```

Redis

State for parallel branches

Replacing state

```
export const ArticleAnnotation = Annotation.Root({
  content: Annotation<string>({
    default: () => ''
    reducer: (prev, next) => next
  })
})
```

- **`default`** — provides an initial value
- **`reducer`** — replaces existing data

State for parallel branches

Merging state

```
export const ArticleAnnotation = Annotation.Root({
  topics: Annotation<string[]>({
    default: () => [],
    reducer: (prev, next) => [...prev, ...next]
  })
})
```

- **`default`** — starts as an empty array
- **`reducer`** — concatenates arrays

Redis

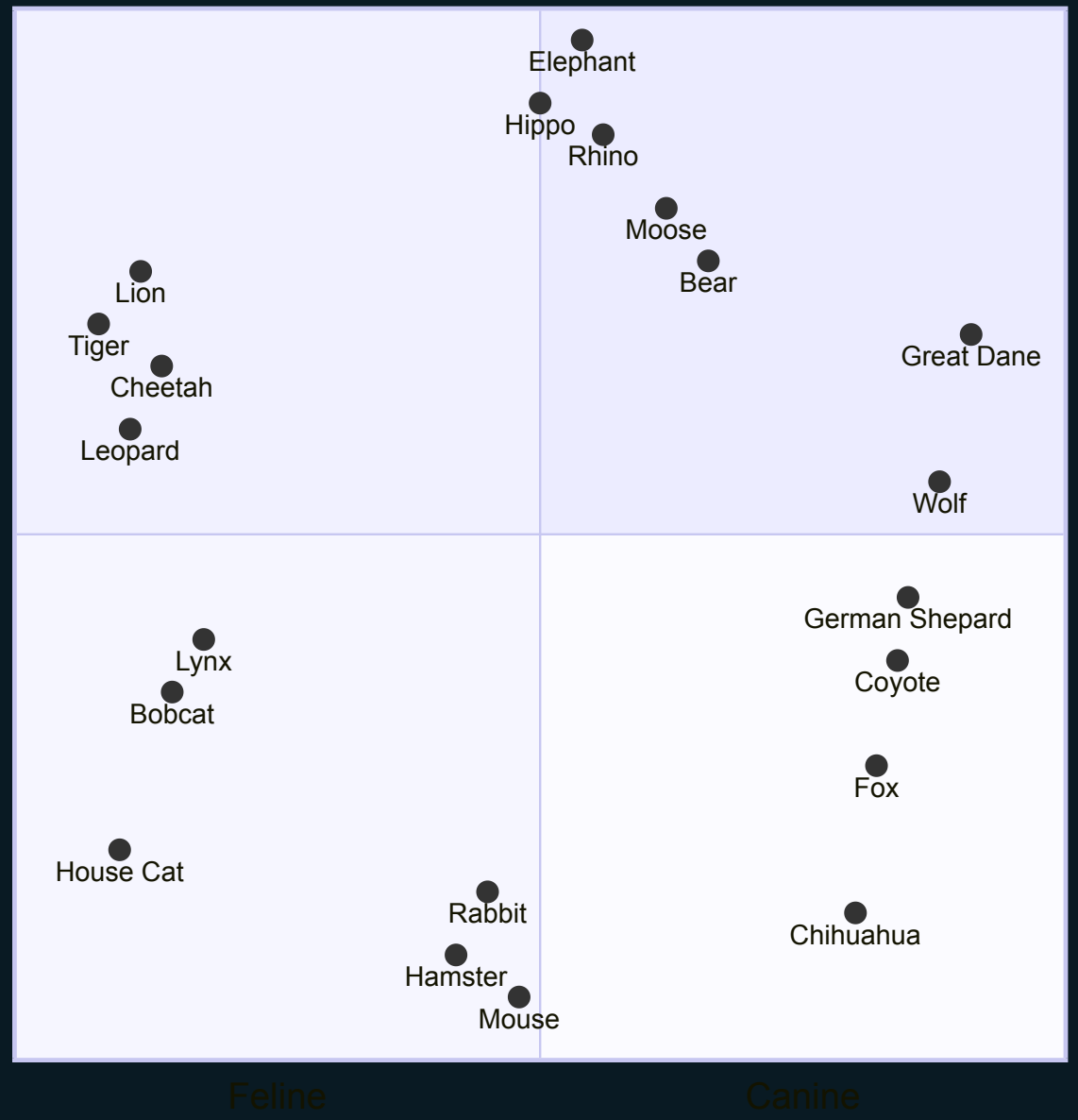
EMBEDDINGS

Redis

What are embeddings?

Numbers that capture meaning

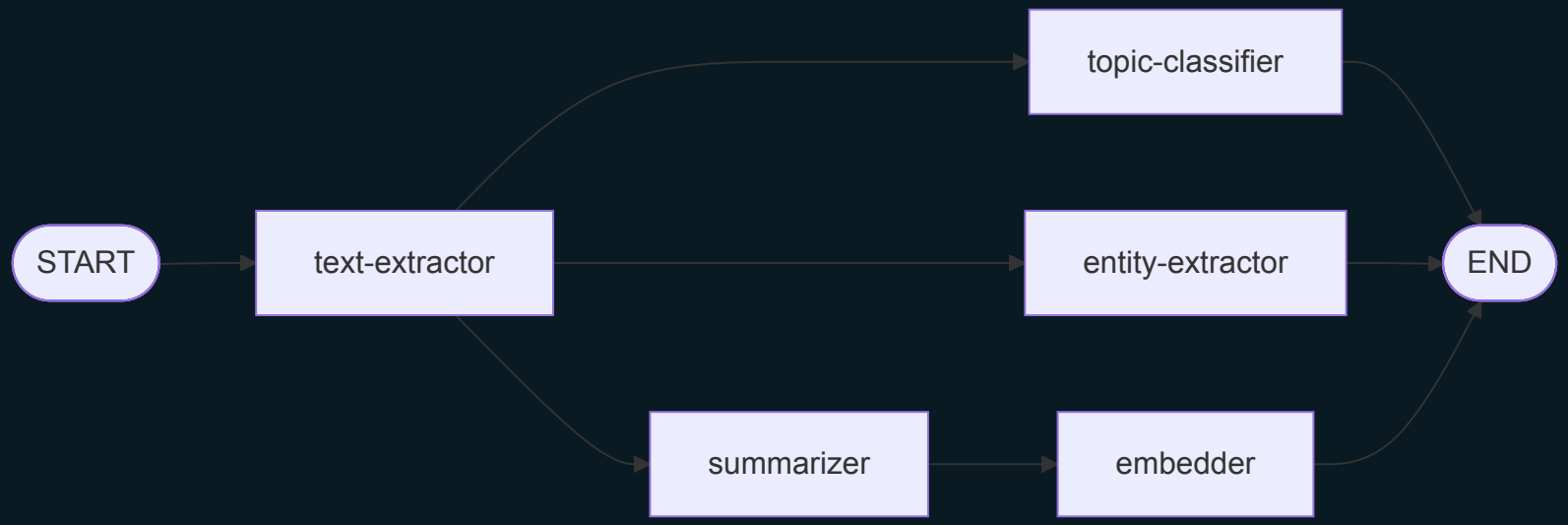
- Unstructured data converted to a vector
- Essentially just coordinates in a high-dimensional space
- Nearby points have similar meanings
- Farther away points have dissimilar meanings



Redis

Embedding the summary

START → text-extractor → summarizer → embedder → END



Redis

The embedder node

Same pattern but uses an embedding model

```
async function embedder(state: ArticleState): Promise<Partial<ArticleState>> {
  const { feedItem, summary } = state

  const textToEmbed = `${feedItem.title}\n\n${summary}`
  const embeddingModel = fetchEmbedder()
  const embedding = await embeddingModel.embedQuery(textToEmbed)

  return { embedding }
}
```

Redis

Fetching the embedding model

Using LangChain.js

```
export function fetchEmbedder(): OpenAIEmbeddings {
  return new OpenAIEmbeddings({
    modelName: 'text-embedding-3-small',
    apiKey: OPENAI_API_KEY
  })
}
```

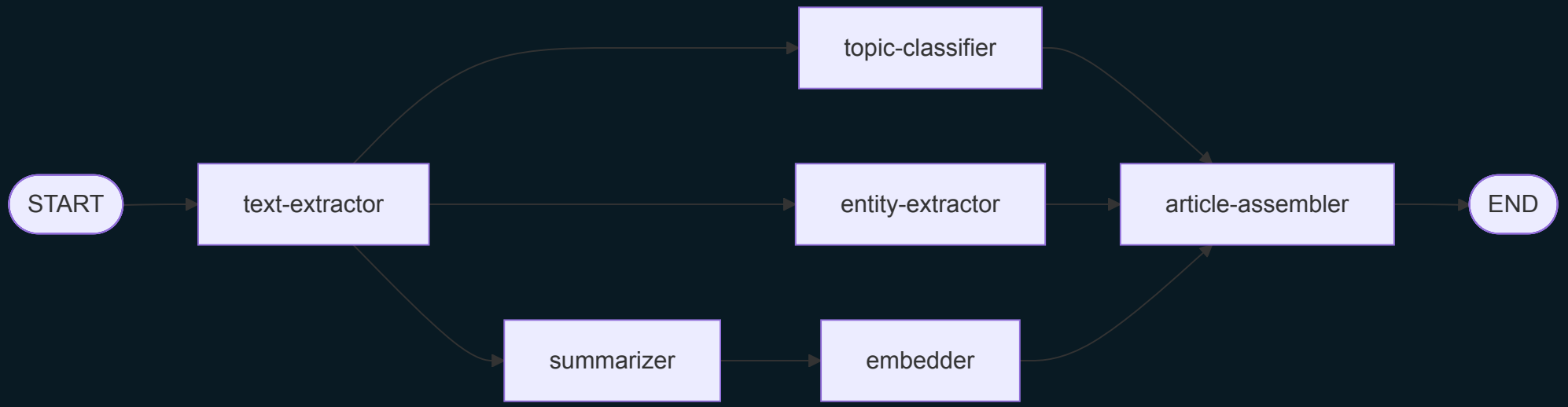
Redis

FAN-IN: ASSEMBLING THE RESULTS

Redis

All roads lead to the assembler

Wait for all branches, then combine



Redis

Deferred nodes

Waits for all incoming edges

```
graph.addNode('article-assembler', articleAssembler, { defer: true })

graph.addEdge('topic-classifier', 'article-assembler')
graph.addEdge('entity-extractor', 'article-assembler')
graph.addEdge('embedder', 'article-assembler')

graph.addEdge('article-assembler', END)
```

- Without `defer`, it runs as soon as the first branch arrives
- With `defer`, it waits for **all** incoming edges

Redis

STORING ARTICLES IN REDIS

Redis

Redis JSON

Native JSON document storage

- Store, retrieve, and update JSON documents in Redis
- Every document lives at a **key** — just like any Redis value
- Access nested fields with **JSONPath** expressions

Redis

Setting and getting documents

JSON.SET and JSON.GET

```
JSON.SET article:42 $ '{"title": "AI News", "topics": ["tech"]}'

JSON.GET article:42
# → {"title": "AI News", "topics": ["tech"]}
```

- $ means ”the root” of the document
- JSON.SET creates or overwrites the document at that key

Redis

JSONPath

Reach into nested fields

```
JSON.GET article:42 $.title
# → ["AI News"]

JSON.GET article:42 $.topics
# → [["tech"]]

JSON.GET article:42 $.topics[0]
# → ["tech"]
```

- `$` = root, `.field` = property, `[n]` = array index
- Always returns an **array** of matches

Redis

In TypeScript

The Node Redis client wraps these commands

```
const key = `${KEY_PREFIX}${id}`
await redis.json.set(key, '$', article)

const doc = await redis.json.get(key)
const topics = await redis.json.get(key, { path: '$.topics' })
```

- `redis.json.set(key, path, value)` — store a document
- `redis.json.get(key)` — retrieve the whole document
- `redis.json.get(key, { path })` — retrieve a nested field

Redis

GO DO IT!

Redis

STAGE 2: ARTICLE SEARCH

Redis as a search engine

Built into Redis — no external service

- Indexes JSON documents **in place**
- No data sync, no replication lag
- Watches for new documents automatically
- TAG, NUMERIC, TEXT, VECTOR, GEO field types

CREATING A SEARCH INDEX

Redis

Defining the schema

```
await redis.ft.create(
  INDEX_NAME,
  {
    '$.source.title': { type: TAG, AS: 'source' },
    '$.topics[*]': { type: TAG, AS: 'topics' },
    '$.publicationDate': { type: NUMERIC, AS: 'date' },
    '$.embedding': {
      type: VECTOR,
      AS: 'embedding',
      ALGORITHM: FLAT,
      TYPE: 'FLOAT32',
      DIM: 1536,
      DISTANCE_METRIC: 'COSINE'
    }
  },
  { ON: 'JSON', PREFIX: KEY_PREFIX }
)
```

Redis

Field types

Each serves a different query style

TYPE	USE CASE	EXAMPLE
TAG	Exact match	`@topics:{AI}`
NUMERIC	Ranges	`@date:[1700000000 +inf]`
VECTOR	Semantic similarity	KNN search
TEXT	Full-text search	Stemming, phonetics
GEO	Radius queries	By coordinates

Redis

STRUCTURED SEARCH

TAG queries

Filter by exact values

```
// AND logic — must match ALL
queryParts.push(`@topics:{AI}`)
queryParts.push(`@people:{Mark Zuckerberg}`)

// OR logic — match ANY
queryParts.push(`@source:{TechCrunch|Ars Technica}`)
```

Redis

NUMERIC queries

Filter by ranges

```
// Date range
queryParts.push(`@date:[${startDate} ${endDate}]`)

// Open-ended
queryParts.push(`@date:[${startDate} +inf]`)
```

Redis

Executing the search

```
const query = queryParts.join(' ') // AND logic
const options = {
  LIMIT: { from: 0, size: limit },
  SORTBY: { BY: 'date', DIRECTION: 'DESC' }
}

const result = await redis.ft.search(INDEX_NAME, query, options)
```

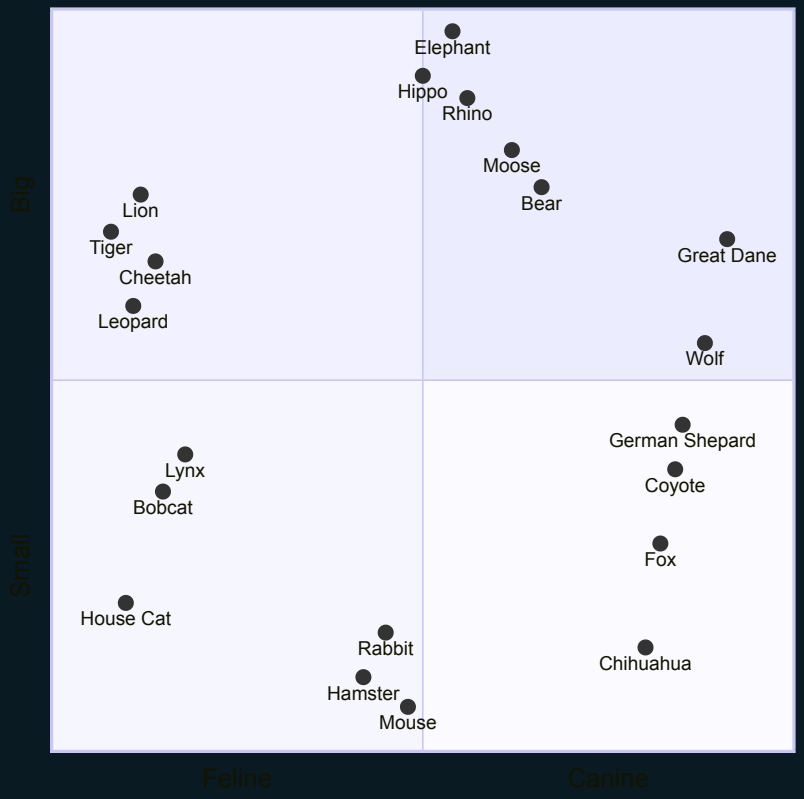
Redis

SEMANTIC SEARCH

Redis

Vector search

Find articles by meaning, not keywords



Redis

K-Nearest Neighbors

```
// Embed the user's query
const embeddingModel = fetchEmbedder()
const embedding = await embeddingModel.embedQuery(query)
const vectorBuffer = Buffer.from(new Float32Array(embedding).buffer)

// Combine structured + vector search
const structuredQuery = queryParts.join(' ')
const vectorQuery = `[KNN ${limit} @embedding $vec]`
const query = `(${textQuery})=>[${semanticQuery}]`

const options = {
  PARAMS: { vec: vectorBuffer },
  SORTBY: { BY: '__embedding_score', DIRECTION: 'ASC' },
  LIMIT: { from: 0, size: limit }
}

const result = await redis.ft.search(INDEX_NAME, query, options)
```

Redis

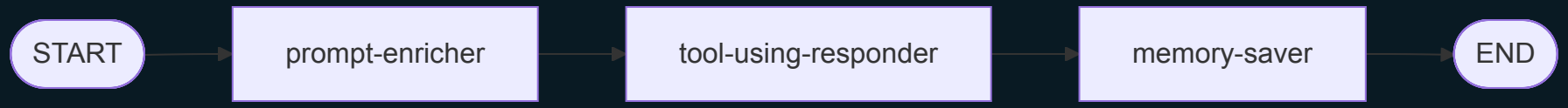
GO DO IT!

Redis

STAGE 3: CHATBOT

The Chatbot workflow

Each node has a specific job



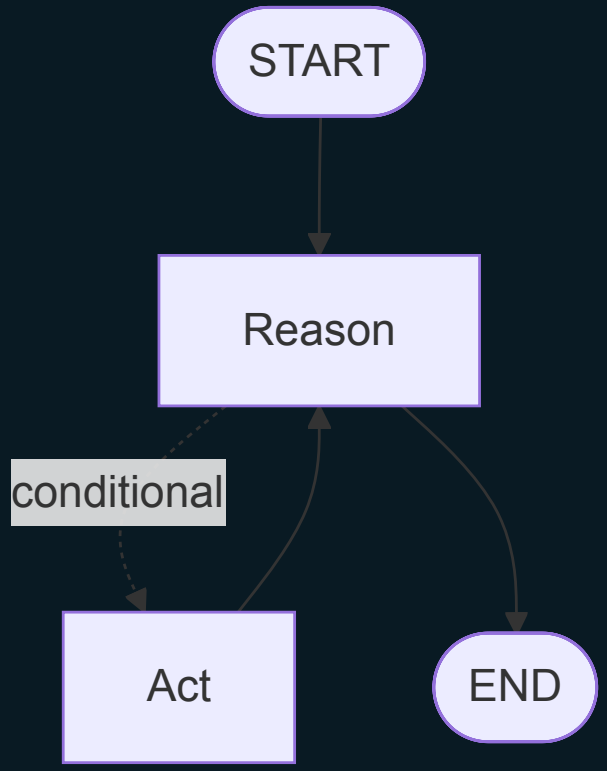
NODE	WHAT IT DOES
prompt-enricher	Fetch memories, build the prompt
tool-using-responder	ReAct agent with search tool
memory-saver	Save the exchange to AMS

TOOLS & THE REACT AGENT

The ReAct pattern

Reasoning + acting

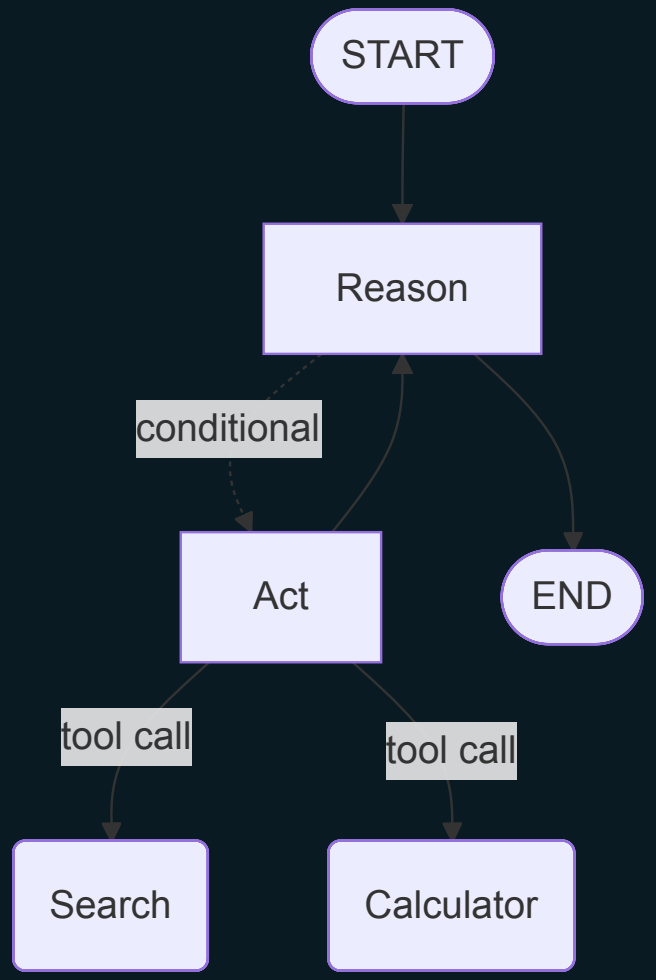
1. **Reason** — What's being asked? What do I know?
2. **Act** — Call a specialized agent to get new information
3. **Observe** — Read the result
4. **Repeat** — Until satisfied



Redis

The ReAct pattern

Calling tools



1. **Reason** — What's being asked? What do I know?
2. **Act** — Call an agent with a tool to do something
3. **Observe** — Read the result
4. **Repeat** — Until satisfied

Redis

What's a tool?

A function the LLM can call

```
function searchArticles(param): Promise<string> { ... }

const searchArticlesTool = tool(searchArticles, {
  name: 'search_articles',
  description: 'Search for news articles...',
  schema: z.object({
    semanticQuery: z.string().optional().describe('Natural language search query'),
    topics: z.array(z.string()).optional().describe('Filter by topics')
    // ...more fields
  })
})
```

- Build it with `tool()` from LangChain.js
- A **schema** defines the parameters

Redis

ReAct Agents in LangGraph.js

Builds the loop for you (but only for tools)

```
const llm = fetchLLM()
const tools = [searchArticlesTool]

const toolUsingResponder = createReactAgent({
  llm,
  tools,
  messageModifier: new SystemMessage(buildPrompt())
})
```

- Give it an LLM, tools, and a system message
- It handles the ReAct loop

Redis

Adding it to the graph

A ReAct agent is just another node

```
graph.addNode('prompt-enricher', promptEnricher)
graph.addNode('tool-using-responder', toolUsingResponder)
graph.addNode('memory-saver', memorySaver)

graph.addEdge(START, 'prompt-enricher')
graph.addEdge('prompt-enricher', 'tool-using-responder')
graph.addEdge('tool-using-responder', 'memory-saver')
graph.addEdge('memory-saver', END)
```

- The `toolUsingResponder` node contains the ReAct agent
- A graph within a graph

AGENT MEMORY SERVER

Redis

Agent Memory Server basics

Two types of memory

Working memory

- Conversation history per session
- Auto-summarizes when it gets long

Long-term memory

- Facts and preferences
- Extracted automatically from working memory
- Persists across sessions
- Retrieved by semantic similarity

All stored in Redis

Redis

The working memory API

REST endpoints on Agent Memory Server

METHOD	ENDPOINT	ACTION
GET	`/v1/working-memory/{sessionId}`	Fetch conversation
PUT	`/v1/working-memory/{sessionId}`	Replace conversation
DELETE	`/v1/working-memory/{sessionId}`	Clear conversation

- PUT replaces the **entire** message list — not an append
- AMS auto-summarizes and extracts long-term memories on PUT

Redis

Saving memory

The memory-saver node

```
async function memorySaver(state: ChatState): Promise<Partial<ChatState>> {
  const { sessionId, userMessage, responseMessage } = state

  const existingMessages = await fetchWorkingMemory(sessionId)
  const updatedMessages = [
    ...existingMessages,
    { role: 'user', content: userMessage },
    { role: 'assistant', content: responseMessage }
  ]

  await updateWorkingMemory(sessionId, { messages: updatedMessages })

  return {}
}
```

Redis

ENRICHING THE PROMPT

Redis

The prompt endpoint

Agent Memory Server assembles the context for you

METHOD	ENDPOINT	ACTION
POST	`/v1/memory/prompt`	Fetch conversation

Returns a collection of messages:

MESSAGE	CONTENT
System message	Conversation summary + long-term memories
Recent messages	Conversation history
Current message	The user's latest input

Redis

Fetch, convert, return

The prompt enricher node

```
async function promptEnricher(state: ChatState): Promise<Partial<ChatState>> {
  const { sessionId, userMessage } = state

  const enrichedPrompt = await fetchMemoryPrompt(sessionId, userMessage)

  const promptMessages = enrichedPrompt.messages.map(msg => {
    if (msg.role === 'user') return new HumanMessage(text)
    if (msg.role === 'assistant') return new AIMessage(text)
    return new SystemMessage(text)
  })

  return { promptMessages }
}
```

Redis

Adding it to the graph

A ReAct agent is just another node

```
graph.addNode('prompt-enricher', promptEnricher)
graph.addNode('tool-using-responder', toolUsingResponder)
graph.addNode('memory-saver', memorySaver)

graph.addEdge(START, 'prompt-enricher')
graph.addEdge('prompt-enricher', 'tool-using-responder')
graph.addEdge('tool-using-responder', 'memory-saver')
graph.addEdge('memory-saver', END)
```

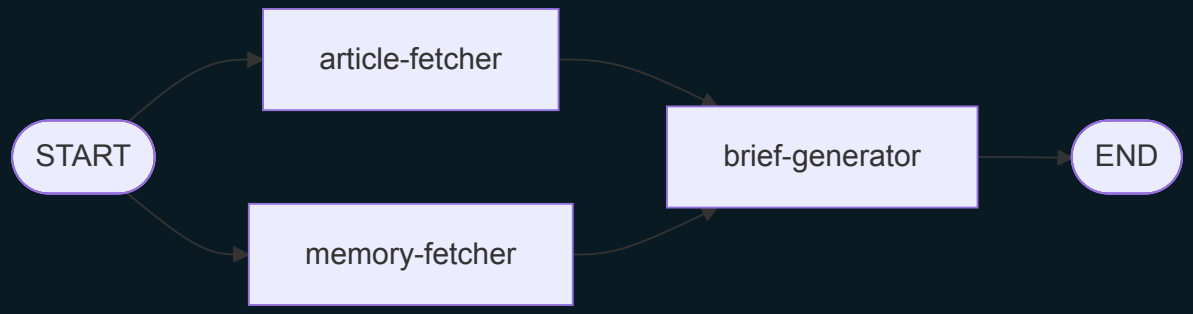
GO DO IT!

Redis

STAGE 4: BRIEF GENERATOR

Bringing it all together

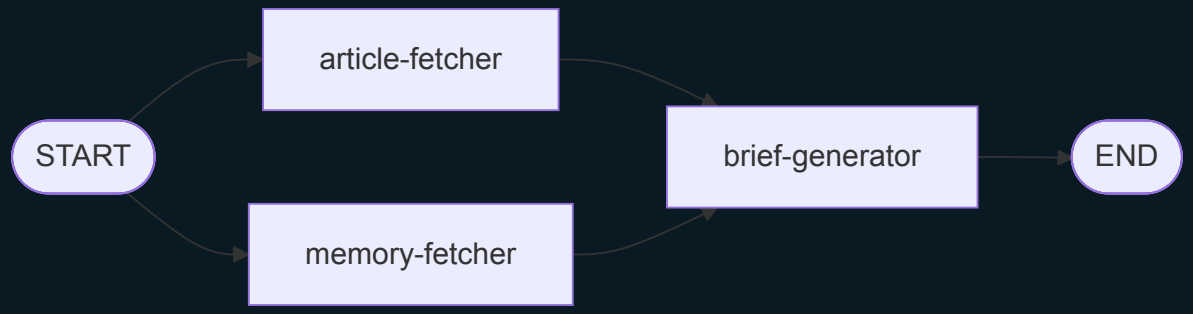
Fetch articles and memories to generate a brief



NODE	WHAT IT DOES
article-fetcher	Fetches recent articles from the database
memory-fetcher	Retrieves long-term memories from AMS
brief-generator	Generates a personalized brief with an LLM

Bringing it all together

Fan-out to fetch



Fan-out from START, fan-in at the generator

Equal-length paths

No defer needed

```
graph.addEdge(START, 'article-fetcher')
graph.addEdge(START, 'memory-fetcher')
graph.addEdge('article-fetcher', 'brief-generator')
graph.addEdge('memory-fetcher', 'brief-generator')
graph.addEdge('brief-generator', END)
```

- Both paths are one node long
- LangGraph.js waits for both automatically
- `defer` is only needed when paths have different lengths

Redis

Memory comes full circle

The chatbot learns, the brief remembers

- **Chat**:
  - ”I’m interested in climate policy”
- **Agent Memory Server**:
  - Stores it in working memory
  - Asynchonously promotes it to a long-term memory
- **Brief generator**:
  - Fetches memories and prioritizes climate articles

Two workflows. Shared memory. One Redis.

Redis

GO DO IT!

Redis

WHAT YOU BUILT

Four stages, one system

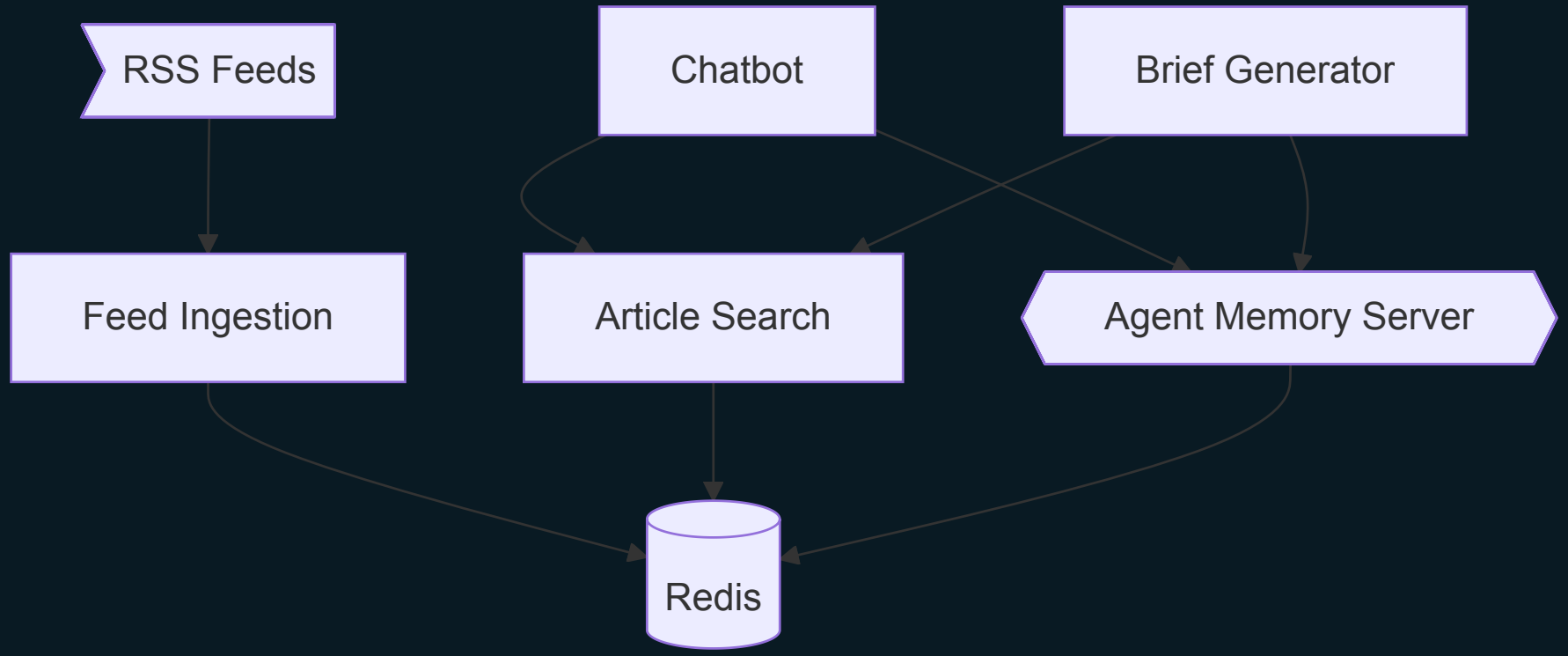
From raw feeds to personalized briefs

	NAME	WHAT WE BUILT
1	Ingestion	RSS → enriched articles in Redis
2	Search	TAG + NUMERIC + VECTOR queries
3	Chat	ReAct agent with tools and memory
4	Brief	Personalized summaries via shared memory

All powered by **Redis**, **LangGraph.js**, and **Agent Memory Server**

Redis

The complete architecture



Redis

Resources

	RESOURCE	LINK
	Workshop Repo	github.com/redis-developer/ai-news-agent-workshop
	LangGraph.js	langchain-ai.github.io/langgraphjs
	Agent Memory Server	redis.github.io/agent-memory-server
	Redis Cloud	redis.io/try-free
	Redis University	university.redis.io
	Discord	discord.gg/redis

Redis

- Guy Royse

SENIOR DEVELOPER
ADVOCATE, REDIS

Socials

@guyroyse

github.com/guyroyse

@guy.dev

guy.dev

Redis

Thank you.